

BS REALTY · DESIGN SYSTEM

Design System

Contributor Guide

The reference pattern for building, exporting, documenting, and publishing components in `@bsrealty/design-system` — with a full worked example.

Package: `@bsrealty/design-system` • Generated: 2026-08-31

BSRealty Design System — Contributor Guide

This document is the reference pattern for how components are built, exported, documented in Storybook, and shipped as the `@bsrealty/design-system` npm package. Follow it for every new component so the package stays consistent and predictable to consume.

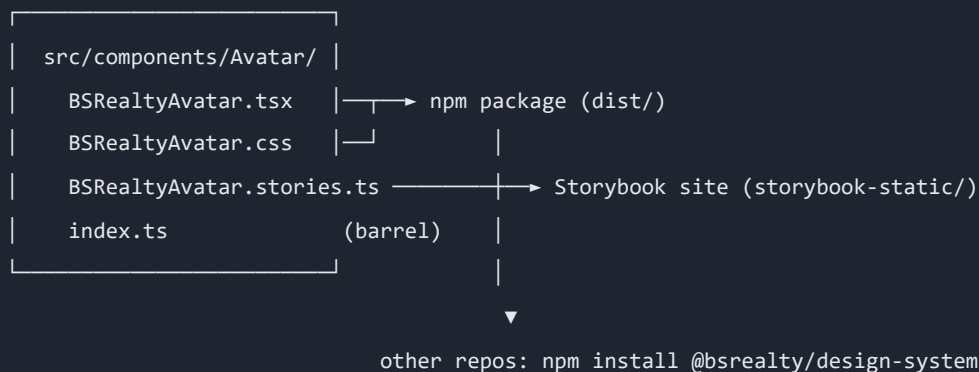
Audience: anyone adding or modifying a component in this repo. **Last worked example added:** `Avatar` (see [Section 7](#)).

1. Mental model — one repo, two outputs

This repository produces two separate, independently-deployed artifacts from the same source:

Output	Built by	Deployed to	Consumed by
npm package (<code>@bsrealty/design-system</code>)	<code>npm run build:lib</code> (<code>vite.lib.config.ts</code>) → <code>dist/</code>	npm registry (or GitHub Packages)	Other codebases, via <code>npm install</code>
Storybook site	<code>npm run build-storybook</code> → <code>storybook-static/</code>	Chromatic / GitHub Pages / Vercel	Designers & developers, in a browser

A component's `.tsx` / `.css` files feed **both** outputs. Its `.stories.tsx` file feeds **only** the Storybook site — story files are explicitly excluded from the npm build (see [Section 5](#)) and never ship to consumers.



2. Folder structure — one folder per component

```
src/  
  components/  
    Avatar/  
      BSRealtyAvatar.tsx      ← component implementation  
      BSRealtyAvatar.css      ← component styles (uses tokens, never hardcodes values)  
      BSRealtyAvatar.stories.ts ← Storybook stories (or .stories.tsx if JSX is needed in args)  
      index.ts                ← barrel: re-exports the component + its prop types  
    Button/  
      ...same shape...  
    ...  
  icons/  
    icons.tsx                 ← shared icon set (used by 2+ components)  
    index.ts  
  tokens/  
    tokens.css                ← design tokens: color, spacing, type scale, radii (single source  
of truth)  
  types/  
    shared.ts                 ← prop-shape fragments reused by 2+ components (Disableable,  
FieldIdentity, ...)  
    index.ts  
  index.ts                    ← THE top-level barrel – the package's public API surface
```

Rule of thumb: everything a component needs to be understood and reused lives in its own folder. Nothing about "Avatar" should require opening any file outside `src/components/Avatar/` except tokens, icons, or shared types — each of which is imported explicitly, never assumed.

3. Anatomy of a component

Every component follows the same four-file shape. Using `Avatar` as the running example (already in this repo — see `src/components/Avatar/`):

3.1 BSRealtyAvatar.tsx — implementation

```
import './BSRealtyAvatar.css';

export type AvatarSize = 'sm' | 'md' | 'lg' | 'xl';

export interface BSRealtyAvatarProps {
  /** Image URL. When omitted (or when it fails to load), falls back to initials. */
  src?: string;
  /** Accessible label / alt text for the image */
  name: string;
  /** Size of the avatar */
  size?: AvatarSize;
}

export const BSRealtyAvatar = ({ src, name, size = 'md' }: BSRealtyAvatarProps) => {
  // ...implementation
};
```

Conventions to follow:

- **Component name:** `BSRealty<Name>` — matches every existing component (`BSRealtyButton` , `BSRealtyTextField` , ...). This prefix is what prevents collisions once this package sits alongside other UI libraries in a consumer's `node_modules` .
- **Props interface name:** `BSRealty<Name>Props` , exported alongside the component.
- **CSS import first, at the top of the file** — a plain side-effect import (`import './BSRealtyAvatar.css'`). This is what makes the CSS travel with the component when it's tree-shaken/deep-imported.
- **JSDoc on every prop** — these comments become the Storybook Controls panel description (via `@storybook/addon-docs` 's autodocs) and the IntelliSense hover text in a consumer's editor. Not optional.
- **Reuse shared fragments** instead of copy-pasting prop fields: if a prop shape already exists in `src/types/shared.ts` (`Disableable` , `FieldIdentity` , `AriaLabelled`), extend it rather than redeclaring `disabled?: boolean` again. Only add a new field there once it's truly identical across 2+ components — see the file's own header comment for the reasoning.
- **Reuse shared icons** from `src/icons/icons.tsx` the same way — check there before inlining a new SVG that might already exist.

3.2 BSRealtyAvatar.css — styles

```
.bsr-avatar {  
  border-radius: var(--bsr-radius-full);  
  background: var(--bsr-color-primary);  
  color: var(--bsr-color-text-inverse);  
  font-family: var(--bsr-font-family-primary);  
}
```

Conventions:

- **Class prefix:** `bsr-` on every class, `bsr-<component>` as the root, `bsr-<component>__<part>` for sub-elements (BEM-ish), `bsr-<component>--<variant>` for variants/sizes. E.g. `bsr-avatar`, `bsr-avatar__image`, `bsr-avatar--lg`.
- **Never hardcode a color, font, spacing, or radius value.** Pull it from `src/tokens/tokens.css` (`var(--bsr-color-primary)`, `var(--bsr-radius-full)`, etc.). If the value you need doesn't exist as a token yet, add it to `tokens.css` first — see that file's header comment. This is what lets a rebrand or dark-mode pass touch one file instead of eleven.
- **A one-line header comment** naming the component and, if it traces to a Figma component, the Figma node reference (see `BSRealtyStarRating.css` for the pattern: `/* ... from Figma Star Ratings component set 270:1313 */`).

3.3 BSRealtyAvatar.stories.ts — Storybook stories

```
import type { Meta, StoryObj } from '@storybook/react-vite';
import { BSRealtyAvatar } from './BSRealtyAvatar';

const meta = {
  title: 'Components/Avatar',
  component: BSRealtyAvatar,
  parameters: { layout: 'centered' },
  tags: ['autodocs'],
  argTypes: {
    size: {
      control: 'select',
      options: ['sm', 'md', 'lg', 'xl'],
      description: 'Avatar size',
    },
  },
};

export satisfies Meta<typeof BSRealtyAvatar>;

export default meta;
type Story = StoryObj<typeof meta>;

export const InitialsMedium: Story = { args: { name: 'Jane Doe', size: 'md' } };
export const WithImage: Story = { args: { name: 'Jane Doe', src: '...', size: 'lg' } };
```

Conventions:

- **title: 'Components/<Name>'** — keeps the Storybook sidebar grouped and matches every existing story file.
- **tags: ['autodocs']** — auto-generates the docs page from your JSDoc comments and **argTypes**. Always include it.
- **One story per meaningfully distinct state** — every size, every variant, every edge case (disabled, error, empty, long text, broken image, etc.), named descriptively (**InitialsLarge** , **BrokenImageFallsBackToInitials**). This is your visual regression + documentation surface at once.
- **.stories.ts vs .stories.tsx** : use plain **.ts** unless a story's **args** needs JSX (e.g. custom render functions, icons as children) — then use **.tsx** . Compare **BSRealtyStarRating.stories.ts** (plain) vs **BSRealtyFAQ.stories.tsx** (needs JSX) in this repo.
- **Use fn() from storybook/test** for callback args (**onClick: fn()**) when the component takes event handlers — see **BSRealtyButton.stories.ts** .

3.4 `index.ts` — the component's barrel

```
export { BSRealtyAvatar } from './BSRealtyAvatar';
export type { BSRealtyAvatarProps, AvatarSize } from './BSRealtyAvatar';
```

This is the **only** file other code should import the component through (aside from Storybook, which imports the `.tsx` directly for HMR reasons). It's also the single place that decides what's public: if a helper type or sub-component shouldn't be consumer-visible, simply don't re-export it here.

4. Wiring a component into the public API

Adding the four files above is not enough — a component isn't part of the published package until it's registered in **two** places:

4.1 `src/index.ts` — the top-level barrel

```
// Components
export * from './components/Avatar';
export * from './components/Button';
// ...
```

This is what makes `import { BSRealtyAvatar } from '@bsrealty/design-system'` work for consumers. Keep the list alphabetical.

4.2 `vite.lib.config.ts` — the build entry map

```
entry: {
  index: resolve(__dirname, 'src/index.ts'),
  Avatar: resolve(__dirname, 'src/components/Avatar/index.ts'),
  Button: resolve(__dirname, 'src/components/Button/index.ts'),
  // ...
},
```

This is what makes the **deep import** `import { BSRealtyAvatar } from '@bsrealty/design-system/Avatar'` possible — each entry becomes its own small bundle in `dist/`, so a consumer who only needs one component doesn't pull in the whole library.

4.3 `package.json` — the `exports` map

```
"/Avatar": {
  "types": "./dist/components/Avatar/index.d.ts",
  "import": "./dist/Avatar.js",
  "require": "./dist/Avatar.cjs"
},
```

This is what makes Node/bundlers actually resolve `@bsrealty/design-system/Avatar` to the right file. The `types` path points into `dist/components/<Name>/index.d.ts` (mirroring `src/`, generated by `vite-plugin-dts`) — **not** a flat `dist/Avatar.d.ts`, which doesn't exist. The `import` / `require` paths point at the flat bundle filenames instead (`dist/Avatar.js` / `dist/Avatar.cjs`), which come from the `fileName` function in `vite.lib.config.ts`. These two naming schemes are different on purpose — don't try to make them match.

Forgetting any one of these three steps is the most common mistake: missing from `src/index.ts` → component is invisible to the main barrel import; missing from `vite.lib.config.ts` → no deep-import bundle gets built; missing from `package.json` → Node refuses to resolve the deep import path even if the bundle exists on disk.

5. The library build

`npm run build:lib` runs `vite build --config vite.lib.config.ts`, which:

1. Compiles every entry in `lib.entry` (Section 4.2) to both ES (`.js`) and CommonJS (`.cjs`) bundles in `dist/`.
2. Generates matching `.d.ts` declaration files via `vite-plugin-dts`, mirroring the `src/` folder structure under `dist/` (so `src/components/Avatar/index.ts` → `dist/components/Avatar/index.d.ts`). Story files (`**/*.stories.*`) and the local demo app (`src/App.tsx`, `src/main.tsx`) are explicitly excluded — see the `dts()` plugin config.
3. Bundles **all** component CSS into one `dist/style.css` (`cssCodeSplit: false`). Consumers import this once, globally.
4. Marks `react` / `react-dom` as external (`peerDependencies`, not bundled) — the consumer's own React instance is used, avoiding duplicate-React bugs.

Run it locally and sanity-check before every publish:

```
npm run build:lib
```

`npm publish` runs this automatically via the `prepublishOnly` script, so a stale `dist/` is never published — but running it manually first lets you catch problems before they hit the registry.

6. Publishing & consuming

6.1 Publish (from this repo)

```
npm version patch # or minor / major — see semver rules below
npm publish --access public
```

Bump the version deliberately:

- **patch** (0.1.0 → 0.1.1) — bug fix, no API change.
- **minor** (0.1.0 → 0.2.0) — new component or new prop, backward compatible.
- **major** (0.1.0 → 1.0.0) — breaking change (renamed/removed prop, changed a component's default behavior, removed a component).

6.2 Consume (in another repo)

```
npm install @bsrealty/design-system
```

```
// once, globally (e.g. in your app's root layout or main.tsx)
import '@bsrealty/design-system/style.css';

// main barrel import
import { BSRealtyAvatar, BSRealtyButton } from '@bsrealty/design-system';

// or a deep import, for a smaller footprint
import { BSRealtyAvatar } from '@bsrealty/design-system/Avatar';
```

6.3 View the catalog (Storybook)

```
npm run storybook # local dev server, http://localhost:6006
npm run build-storybook # static site → storybook-static/, deploy anywhere
```

7. Worked example — adding a new `Avatar` component, step by step

This section walks through exactly what was done to add `Avatar` to this repo, in the order it was done. The result is already committed in `src/components/Avatar/` — use it as a live reference alongside this guide, and repeat these same steps for the next new component.

Component being added: an `Avatar` — shows a person's photo, or their initials as a fallback when no photo is available or the image fails to load. Sizes: `sm` / `md` / `lg` / `xl` .

Step 1 — create the component folder

```
mkdir -p src/components/Avatar
```

Step 2 — write the component (`BSRealtyAvatar.tsx`)

```
import './BSRealtyAvatar.css';

export type AvatarSize = 'sm' | 'md' | 'lg' | 'xl';

export interface BSRealtyAvatarProps {
  /** Image URL. When omitted (or when it fails to load), falls back to initials. */
  src?: string;
  /** Accessible label / alt text for the image */
  name: string;
  /** Size of the avatar */
  size?: AvatarSize;
}

const getInitials = (name: string): string =>
  name
    .trim()
    .split(/\s+/)
    .slice(0, 2)
    .map((part) => part[0]?.toUpperCase() ?? '')
    .join('');

export const BSRealtyAvatar = ({ src, name, size = 'md' }: BSRealtyAvatarProps) => {
  return (
    <span className={['bsr-avatar', `bsr-avatar--${size}`].join(' ')}>
      {src ? (
        <img
          className="bsr-avatar__image"
          src={src}
          alt={name}
          onError={(e) => {
            e.currentTarget.style.display = 'none';
          }}
        />
      ) : null}
      <span className="bsr-avatar__initials" aria-hidden={!src}>
        {getInitials(name)}
      </span>
    </span>
  );
};
```

Notice this follows every convention from Section 3.1: `BSRealtyAvatar` naming, `BSRealtyAvatarProps` export, CSS side-effect import at the top, JSDoc on every prop.

Step 3 — write the styles (`BSRealtyAvatar.css`), using tokens only

```
/* BS Realty Avatar */

.bsr-avatar {
  position: relative;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
  border-radius: var(--bsr-radius-full);
  background: var(--bsr-color-primary);
  color: var(--bsr-color-text-inverse);
  font-family: var(--bsr-font-family-primary);
  font-weight: var(--bsr-font-weight-semibold);
  overflow: hidden;
  user-select: none;
}

.bsr-avatar__image {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  object-fit: cover;
}

.bsr-avatar--sm { width: 24px; height: 24px; font-size: var(--bsr-font-size-label-small); }
.bsr-avatar--md { width: 32px; height: 32px; font-size: var(--bsr-font-size-label-medium); }
.bsr-avatar--lg { width: 48px; height: 48px; font-size: var(--bsr-font-size-body-medium); }
.bsr-avatar--xl { width: 64px; height: 64px; font-size: var(--bsr-font-size-title-medium); }
```

Every color, radius, font, and weight is a `var(--bsr-*)` token from `src/tokens/tokens.css` — zero hardcoded values. Only `width` / `height` / `overflow` (structural, not brand-dependent) are literal.

Step 4 — write the barrel (`index.ts`)

```
export { BSRealtyAvatar } from './BSRealtyAvatar';
export type { BSRealtyAvatarProps, AvatarSize } from './BSRealtyAvatar';
```

Step 5 — write the Storybook stories (`BSRealtyAvatar.stories.ts`)

```
import type { Meta, StoryObj } from '@storybook/react-vite';
import { BSRealtyAvatar } from './BSRealtyAvatar';

const meta = {
  title: 'Components/Avatar',
  component: BSRealtyAvatar,
  parameters: { layout: 'centered' },
  tags: ['autodocs'],
  argTypes: {
    name: { control: 'text', description: 'Full name – used for alt text and the initials fallback' },
    src: { control: 'text', description: 'Image URL (omit to show initials only)' },
    size: { control: 'select', options: ['sm', 'md', 'lg', 'xl'], description: 'Avatar size' },
  },
} satisfies Meta<typeof BSRealtyAvatar>;

export default meta;
type Story = StoryObj<typeof meta>;

export const InitialsSmall: Story = { args: { name: 'Jane Doe', size: 'sm' } };
export const InitialsMedium: Story = { args: { name: 'Jane Doe', size: 'md' } };
export const InitialsLarge: Story = { args: { name: 'Jane Doe', size: 'lg' } };
export const InitialsXL: Story = { args: { name: 'Jane Doe', size: 'xl' } };
export const SingleName: Story = { args: { name: 'Cher', size: 'lg' } };
export const WithImage: Story = { args: { name: 'Jane Doe', src: 'https://i.pravatar.cc/128?img=47', size: 'lg' } };
export const BrokenImageFallsBackToInitials: Story = {
  args: { name: 'Jane Doe', src: 'https://example.invalid/broken.jpg', size: 'lg' },
};
```

Every size gets its own story, plus the two edge cases that matter for this component: a single-word name (initials logic), and a broken image URL (fallback behavior).

At this point you can already run `npm run storybook` and see Avatar rendered and documented in the sidebar under **Components/Avatar** — but it is **not yet** part of the published npm package. That's steps 6–7.

Step 6 — register it in the top-level barrel

Edit `src/index.ts` :

```
// Components
+export * from './components/Avatar';
export * from './components/Button';
export * from './components/FAQ';
...
```

Step 7 — register it as a library build entry

Edit `vite.lib.config.ts` :

```
entry: {
  index: resolve(__dirname, 'src/index.ts'),
+ Avatar: resolve(__dirname, 'src/components/Avatar/index.ts'),
  Button: resolve(__dirname, 'src/components/Button/index.ts'),
  ...
},
```

Step 8 — add the deep-import path to `package.json`

Edit the `"exports"` map in `package.json` :

```
"exports": {
  ".": { ... },
+ "./Avatar": {
+   "types": "./dist/components/Avatar/index.d.ts",
+   "import": "./dist/Avatar.js",
+   "require": "./dist/Avatar.cjs"
+ },
  "./Button": { ... },
  ...
}
```

Step 9 — verify

Run all three checks before considering the component done:

```
# 1. No new TypeScript errors
npx tsc --noEmit -p tsconfig.app.json

# 2. Library build succeeds and produces the expected files
npm run build:lib
ls dist/Avatar.js dist/Avatar.cjs dist/components/Avatar/index.d.ts

# 3. Storybook still builds cleanly with the new story included
npm run build-storybook
```

Then a runtime smoke test that both import paths actually resolve:

```
node -e "
  console.log(Object.keys(require('./dist/index.cjs')).includes('BSRealtyAvatar')); // true
  console.log(Object.keys(require('./dist/Avatar.cjs'))); //
  ['BSRealtyAvatar']
"
```

Step 10 — commit

```
git add src/components/Avatar src/index.ts vite.lib.config.ts package.json
git commit -m "feat: add BSRealtyAvatar component"
```

That's the full loop. The next new component — `Badge` , `Modal` , `Tooltip` , whatever's next — follows these exact ten steps.

8. Quick checklist (copy this for every new component)

- ☐ `mkdir src/components/<Name>`
- ☐ `BSRealty<Name>.tsx` — implementation, JSDoc'd props, imports its own `.css`
- ☐ `BSRealty<Name>.css` — `bsr-` prefixed classes, tokens only, no hardcoded values
- ☐ `index.ts` — barrel exporting the component + its prop types
- ☐ `BSRealty<Name>.stories.ts(x)` — one story per meaningful state, `tags: ['autodocs']`
- ☐ `src/index.ts` — add `export * from './components/<Name>';` (alphabetical)
- ☐ `vite.lib.config.ts` — add the entry under `lib.entry`
- ☐ `package.json` — add the `"./<Name>"` block under `"exports"`
- ☐ `npx tsc --noEmit -p tsconfig.app.json` passes
- ☐ `npm run build:lib` succeeds, expected `dist/` files exist
- ☐ `npm run build-storybook` succeeds

- ☐ Runtime smoke test: both barrel and deep imports resolve